

Software Requirements Specification (SRS)

Document Summary Assistant (DocuSense AI)
IEEE Std 830-1998 / ISO/IEC/IEEE 29148 Compliant Specification

Advait Varhade

Software Engineering Team

GitHub Repository: <https://github.com/AdvaitVarhade/docusense-ai>

August 24, 2026

Abstract

Abstract—This Software Requirements Specification (SRS) establishes the formal engineering requirements for DocuSense AI, an intelligent full-stack document intelligence and summarization platform. Engineered using a Hexagonal (Ports & Adapters) Architecture on Next.js 15 App Router, TypeScript, and Tailwind CSS, the system incorporates a Tri-Tier Adaptive OCR extraction pipeline and real-time Server-Sent Events (SSE) AI synthesis with an in-stream multi-model fallback chain. This document details the functional, non-functional, interface, and security requirements, concluding with a day-wise phased execution timeline spanning August 19, 2026 to August 24, 2026.

Keywords Software Requirements Specification, IEEE 830, Next.js 15, Hexagonal Architecture, Document Intelligence, Tri-Tier OCR, SSE Streaming, LLM Fallback Chain.

1. Introduction

This document specifies the software requirements for DocuSense AI in compliance with the IEEE Std 830-1998 standard.

1.1. Purpose & Scope

DocuSense AI ingests heterogeneous digital PDFs, scanned contracts, receipts, and images, performs layout-aware text extraction via WebAssembly and Optical Character Recognition (OCR), and generates multi-fidelity streaming summaries (*Short*, *Medium*, *Long*) paired with highlighted key takeaways and actionable improvement critiques.

2. Overall System Description

2.1. Hexagonal Clean Architecture

The system enforces strict decoupling across four primary layers:

- **Domain Layer** (`src/domain/`): Pure business entities, Zod validation schemas, and port interfaces (`IExtractionEngine`, `ISummarizationEngine`).
- **Application Layer** (`src/application/`): Orchestration use-cases (`ExtractDocumentUseCase`, `SummarizeDocumentUseCase`) and domain services.
- **Infrastructure Layer** (`src/infrastructure/`): Concrete external adapters for `unpdf` (WASM PDF), `tesseract.js` (WASM OCR), Gemini 3.x APIs, and offline mock engines.
- **Presentation Layer** (`src/components/`): Next.js 15 UI and Server-Sent Events (SSE) route handlers.

3. Functional Requirements

3.1. Ingestion & Tri-Tier Extraction (FR-1 & FR-2)

- **FR-1.1**: Accepts PDF, PNG, JPG, JPEG, WEBP, and TIFF up to 25MB via drag-and-drop or file picker.
- **FR-2.1 (Digital PDF)**: Executes in-memory `unpdf` WebAssembly parsing in $< 50\text{ms}$ with zero AI token cost.
- **FR-2.2 (Scanned Image OCR)**: Executes sandboxed `tesseract.js` WebAssembly OCR locally with zero external API calls.
- **FR-2.3 (Telemetry)**: Automatically computes word count, character count, estimated reading time, and confidence metrics.

3.2. AI Summarization & Streaming (FR-3 & FR-4)

- **FR-3.1 (Multi-Fidelity Presets)**: Supports Short ($\sim 150\text{w}$ TL;DR), Medium ($\sim 400\text{w}$ Thematic Synthesis), and Long ($\sim 900\text{w}$ Deep-Dive) modes.
- **FR-4.1 (SSE Streaming)**: Streams tokens in real-time over HTTP Server-Sent Events with $< 450\text{ms}$ Time-to-First-Token.
- **FR-4.2 (Stop Control)**: Enables immediate user cancellation via `AbortController`.

3.3. Takeaways, Critiques & Export Suite (FR-5 & FR-6)

- **FR-5.1:** Extracts categorized Key Takeaways (*metric, strategic, operational, risk*).
- **FR-5.2:** Generates categorized Improvement Critiques (*clarity, structure, completeness, actionable*).
- **FR-6.1:** Provides 1-click client-side export to Markdown (.md), formatted JSON (.json), clean text (.txt), and system clipboard.

4. Non-Functional Requirements (NFRs)

- **NFR-1 (Performance):** Time-to-First-Token < 450ms; PDF extraction < 100ms; Shared First Load JS < 110kB.
- **NFR-2 (Security):** All raw inputs enclosed within strict `<document_content>` XML barriers; output markdown sanitized via `rehype-sanitize`.
- **NFR-3 (Privacy):** In-memory ephemeral processing with zero persistent disk storage.
- **NFR-4 (Reliability):** Inside-stream fallback loop (`gemini-3.6-flash` → `gemini-3.5-flash-lite` → `mock_offline_engine`) ensuring zero unhandled crashes.

5. Project Timeline & Phased Implementation Plan (August 19 – August 24, 2026)

The project was executed in a 6-day phased development lifecycle from August 19 to August 24, 2026:

6. Verification Results

The application achieved a **100% pass rate** across 9 automated test suites and 90 unit/e2e test cases in Vitest. Production build compiled cleanly with zero errors.

Date	Phase Scope & Key Deliverables
Aug 19	Phase 1: Architecture & Foundation Formulated PRD, authored ADR-001/002/003, scaffolded Next.js 15 Hexagonal architecture.
Aug 20	Phase 2: Ingestion & Tri-Tier OCR Pipeline Implemented <code>/api/extract</code> , <code>unpdf</code> WASM parser, <code>tesseract.js</code> OCR, and <code>DocumentUploader.tsx</code> .
Aug 21	Phase 3: AI Engine & SSE Streaming Implemented <code>/api/summarize</code> SSE endpoint, prompt injection shields, and multi-fidelity presets.
Aug 22	Phase 4: Dashboard UI & Export Suite Built <code>SummaryViewer.tsx</code> , length switcher, and 4-format client-side export suite (.md, .json, .txt, clipboard).
Aug 23	Phase 5: Multi-Model Chain & Resilience Configured Gemini 3.x fallback routing (<code>gemini-3.6-flash</code> , <code>gemini-3.5-flash-lite</code>), stream catchers, and bug fixes.
Aug 24	Phase 6: Full Verification & Public Release 90/90 Vitest tests passed (100%), compiled IEEE SRS specification, and deployed to GitHub repository.

Table 1: Day-Wise Chronological Development Schedule